

MAINHEAD Backfill Assessment

Production-safe assessment for populating the structured `SiteVisit.mainheadId` foreign key from the deprecated `SiteVisit.mainhead` text column, so that legacy visits become visible to QA actors under the Governance G3 scope. **Assessment only — no database changes are executed.**

CONTEXT — The G3 QA scope predicate is `{ mainheadId: { in: qaMainheadIds } }` (`site-visits.service.ts:2165`) and consults the structured FK only. Authorization is correct; the remaining gap is data shape: legacy rows carry `mainheadId = NULL` and hold their MAINHEAD identity only in the text column. This backfill closes that gap.

SAFETY POSTURE — The backfill is idempotent (fills NULL only, never reassigns), ambiguity-guarded (a legacy text is applied only when exactly one active Mainhead matches), reversible (exact rollback from a backup table), and leaves `updatedAt` untouched so last-activity and operational-health staleness signals are not perturbed.

0. Grounding facts (verified against the repo)

Object	Physical name	Notes
Structured FK	"SiteVisit"."mainheadId" UUID NULL	FK <code>SiteVisit_mainheadId_fkey</code> to "Mainhead"("id"), ON DELETE SET NULL.
Legacy text	"SiteVisit"."mainhead" TEXT NULL	Deprecated. May hold a name, a code, a label, or free text from mobile.
Tenant anchor	"SiteVisit"."tenantId" UUID NOT NULL	Present on the row directly.
Target	"Mainhead": id, code (nullable), name, isActive	No unique constraint on code or name — ambiguity is possible and is guarded.

Legacy text origin (`site-visits.service.ts:460`): `mainhead = normalize(dto.mainhead) ?? (Mainhead.name ?? Mainhead.code ?? workPackage.mainhead)`. Matching is therefore case-insensitive, whitespace-normalized, and tested against name, code, and the code - name label. Note: neither Mainhead nor Organization carries a `tenantId`; the pilot is single-tenant, so text matching is safe. The multi-tenant guard is documented in §6.

1. Exact rows with `SiteVisit.mainheadId = NULL`

Read-only enumeration of every NULL-FK row with its legacy text and the candidate it would resolve to. Each row is classified `MATCH_UNIQUE` (safe), `AMBIGUOUS` (never auto-assigned), `UNMATCHED`, or `NO_LEGACY_TEXT`.

```
-- D1: Exact NULL-FK rows, with legacy text and candidate resolution preview
WITH legacy AS (
  SELECT
    sv.id,
    sv."tenantId",
    sv."mainhead" AS legacy_text,
    lower(btrim(regexp_replace(sv."mainhead", '\s+', ' ', 'g'))) AS norm_text,
    sv.status,
    sv."startedAt"
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL
),
candidate AS (
  SELECT l.id, m.id AS candidate_mainhead_id, m.code AS candidate_code, m.name AS candidate_name
  FROM legacy l
  JOIN "Mainhead" m
```

```

    ON m."isActive" = TRUE
    AND l.norm_text IS NOT NULL AND l.norm_text <> ''
    AND (
        lower(btrim(regexp_replace(m.name, '\s+', ' ', 'g'))) = l.norm_text
        OR lower(btrim(regexp_replace(coalesce(m.code, ''), '\s+', ' ', 'g'))) = l.norm_text
        OR lower(btrim(regexp_replace(coalesce(m.code, '') || ' - ' || m.name, '\s+', ' ', 'g'))) = l.norm_text
    )
)
SELECT
    l.id AS site_visit_id,
    l."tenantId" AS tenant_id,
    l.legacy_text,
    l.status,
    l."startedAt",
    count(c.candidate_mainhead_id) AS candidate_count,
    min(c.candidate_code) AS sample_candidate_code,
    min(c.candidate_name) AS sample_candidate_name,
    CASE
        WHEN count(DISTINCT c.candidate_mainhead_id) = 1 THEN 'MATCH_UNIQUE'
        WHEN count(DISTINCT c.candidate_mainhead_id) > 1 THEN 'AMBIGUOUS'
        WHEN l.legacy_text IS NULL OR btrim(l.legacy_text) = '' THEN 'NO_LEGACY_TEXT'
        ELSE 'UNMATCHED'
    END AS resolution
FROM legacy l
LEFT JOIN candidate c ON c.id = l.id
GROUP BY l.id, l."tenantId", l.legacy_text, l.status, l."startedAt"
ORDER BY resolution, l."startedAt";

```

2. Mapping: legacy text to Mainhead records

Distinct mapping table for human review **before** anything runs — this is the production-safety gate. Only rows with decision = WILL_BACKFILL are touched; AMBIGUOUS_SKIP and UNMATCHED_SKIP are reported and left for a human decision.

```

-- D2: Distinct legacy-text -> Mainhead mapping with ambiguity classification
WITH legacy AS (
    SELECT
        sv."mainhead" AS legacy_text,
        lower(btrim(regexp_replace(sv."mainhead", '\s+', ' ', 'g'))) AS norm_text
    FROM "SiteVisit" sv
    WHERE sv."mainheadId" IS NULL
        AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
distinct_text AS (
    SELECT legacy_text, norm_text, count(*) AS affected_rows
    FROM legacy GROUP BY legacy_text, norm_text
),
matched AS (
    SELECT
        d.legacy_text, d.affected_rows,
        m.id AS mainhead_id, m.code AS mainhead_code, m.name AS mainhead_name,
        CASE
            WHEN lower(btrim(regexp_replace(m.name, '\s+', ' ', 'g'))) = d.norm_text THEN 'name'
            WHEN lower(btrim(regexp_replace(coalesce(m.code, ''), '\s+', ' ', 'g'))) = d.norm_text THEN 'code'
            WHEN lower(btrim(regexp_replace(coalesce(m.code, '') || ' - ' || m.name, '\s+', ' ', 'g'))) = d.norm_text THEN
'label'
        END AS matched_on
    FROM distinct_text d
    JOIN "Mainhead" m
        ON m."isActive" = TRUE
        AND (
            lower(btrim(regexp_replace(m.name, '\s+', ' ', 'g'))) = d.norm_text
            OR lower(btrim(regexp_replace(coalesce(m.code, ''), '\s+', ' ', 'g'))) = d.norm_text
            OR lower(btrim(regexp_replace(coalesce(m.code, '') || ' - ' || m.name, '\s+', ' ', 'g'))) = d.norm_text
        )
)
SELECT
    d.legacy_text,
    d.affected_rows,
    count(DISTINCT m.mainhead_id) AS distinct_mainhead_matches,

```

```

    string_agg(DISTINCT m.mainhead_code || ' / ' || m.mainhead_name || ' (' || m.matched_on || ')', ';' ) AS
matched_mainheads,
CASE
    WHEN count(DISTINCT m.mainhead_id) = 1 THEN 'WILL_BACKFILL'
    WHEN count(DISTINCT m.mainhead_id) > 1 THEN 'AMBIGUOUS_SKIP'
    ELSE 'UNMATCHED_SKIP'
END AS decision
FROM distinct_text d
LEFT JOIN matched m ON m.legacy_text = d.legacy_text
GROUP BY d.legacy_text, d.affected_rows
ORDER BY decision, d.affected_rows DESC;

```

3. Exact SQL migration / backfill script

Transactional, idempotent, ambiguity-guarded, with a backup table for exact rollback. Run inside a maintenance window after §1/§2 have been reviewed. The in-transaction check (3d) auto-rolls-back on any integrity failure before commit.

```
-- D3: Production-safe MAINHEAD backfill for SiteVisit
-- Idempotent | ambiguity-guarded | self-documenting backup table | single transaction
BEGIN;

-- 3a. Backup / audit table (forward record + rollback source). Survives the transaction.
CREATE TABLE IF NOT EXISTS "_backfill_SiteVisit_mainhead_20260604" (
  site_visit_id uuid PRIMARY KEY,
  previous_mainhead_id uuid, -- always NULL for these rows; captured for audit
  legacy_mainhead_text text,
  assigned_mainhead_id uuid NOT NULL,
  backfilled_at timestamptz NOT NULL DEFAULT now()
);

-- 3b. Resolve the unique-match set (same logic as D2; only 1-distinct-candidate rows qualify).
WITH legacy AS (
  SELECT sv.id, sv."mainhead" AS legacy_text,
         lower(btrim(regexp_replace(sv."mainhead", '\s+', ' ', 'g'))) AS norm_text
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL
        AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
candidate AS (
  SELECT l.id, l.legacy_text, m.id AS mainhead_id
  FROM legacy l
  JOIN "Mainhead" m
    ON m."isActive" = TRUE
  AND (
    lower(btrim(regexp_replace(m.name, '\s+', ' ', 'g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code, ''), '\s+', ' ', 'g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code, '') || ' - ' || m.name, '\s+', ' ', 'g'))) = l.norm_text
  )
),
resolved AS (
  SELECT id, legacy_text,
         count(DISTINCT mainhead_id) AS match_count,
         min(mainhead_id) AS mainhead_id -- meaningful only when match_count = 1
  FROM candidate GROUP BY id, legacy_text
)
INSERT INTO "_backfill_SiteVisit_mainhead_20260604"
  (site_visit_id, previous_mainhead_id, legacy_mainhead_text, assigned_mainhead_id)
SELECT r.id, NULL::uuid, r.legacy_text, r.mainhead_id
FROM resolved r
WHERE r.match_count = 1
ON CONFLICT (site_visit_id) DO NOTHING; -- re-run safe

-- 3c. Apply the backfill ONLY to captured rows, ONLY while still NULL.
-- NOTE: updatedAt is intentionally NOT modified (preserves last-activity / staleness signals).
UPDATE "SiteVisit" sv
SET "mainheadId" = b.assigned_mainhead_id
FROM "_backfill_SiteVisit_mainhead_20260604" b
WHERE sv.id = b.site_visit_id
      AND sv."mainheadId" IS NULL; -- idempotency guard

-- 3d. In-transaction sanity check: every touched row points to an ACTIVE Mainhead.
DO $$
DECLARE bad int;
BEGIN
  SELECT count(*) INTO bad
  FROM "_backfill_SiteVisit_mainhead_20260604" b
  JOIN "SiteVisit" sv ON sv.id = b.site_visit_id
  LEFT JOIN "Mainhead" m ON m.id = sv."mainheadId" AND m."isActive" = TRUE
  WHERE sv."mainheadId" <> b.assigned_mainhead_id -- diverged from plan
        OR m.id IS NULL; -- missing/inactive Mainhead
  IF bad > 0 THEN
    RAISE EXCEPTION 'Backfill integrity check failed for % row(s); rolling back', bad;
  END IF;
END;
```

```
END $$;

COMMIT;
-- Keep the backup table until post-deploy validation passes, then archive/drop.
```

To ship as a Prisma migration, save 3b–3c (without the backup table) under `prisma/migrations/<ts>_governance_g3_backfill_sitevisit_mainhead/migration.sql`. Prisma migrations are forward-only and cannot hold the rollback table, so the reviewed manual transaction above is the recommended production path.

4. Rollback strategy

Primary — exact reversal from the backup table. Reverts only rows the backfill set, and only if they still hold the assigned value, so it never clobbers a later legitimate edit.

```
-- D4a: Exact reversal from the backup table (preferred).
-- Reverts ONLY rows the backfill set, and ONLY if they still hold the assigned value.
BEGIN;
UPDATE "SiteVisit" sv
SET "mainheadId" = b.previous_mainhead_id -- NULL
FROM "_backfill_SiteVisit_mainhead_20260604" b
WHERE sv.id = b.site_visit_id
      AND sv."mainheadId" = b.assigned_mainhead_id; -- guard: only undo our own writes
COMMIT;
-- Optional, once confirmed reverted:
-- DROP TABLE "_backfill_SiteVisit_mainhead_20260604";
```

Fallback (no backup table, Prisma-migration path). Every targeted row's prior value was provably NULL, so reversal is "set back to NULL where the FK now matches a legacy-text resolution" — less precise than the primary path, which is preferred. **Blast radius:** the FK is ON DELETE SET NULL and rollback only writes NULL, so there is no cascade, no row deletion, and no dependent-record impact; inspections and defects change only in *visibility*, not structure.

5. Validation queries — before and after

Before (baseline)

```
-- B1: NULL-FK volume and tenant total
SELECT
  count(*) AS total_visits,
  count(*) FILTER (WHERE "mainheadId" IS NULL) AS null_fk_visits,
  count(*) FILTER (WHERE "mainheadId" IS NOT NULL) AS populated_fk_visits
FROM "SiteVisit";

-- B2: planned coverage (must equal the WILL_BACKFILL totals from D2)
WITH legacy AS (
  SELECT sv.id, lower(btrim(regexp_replace(sv."mainhead", '\s+', ' ', 'g'))) AS norm_text
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
cand AS (
  SELECT l.id, count(DISTINCT m.id) AS mc
  FROM legacy l
  LEFT JOIN "Mainhead" m ON m."isActive" AND (
    lower(btrim(regexp_replace(m.name, '\s+', ' ', 'g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code, ''), '\s+', ' ', 'g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code, '') || ' - ' || m.name, '\s+', ' ', 'g'))) = l.norm_text
  )
  GROUP BY l.id
)
SELECT
  count(*) FILTER (WHERE mc = 1) AS will_backfill,
  count(*) FILTER (WHERE mc > 1) AS ambiguous_skip,
  count(*) FILTER (WHERE mc = 0) AS unmatched_skip
FROM cand;

-- B3: QA baseline. Replace the id list with the qaMainheadIds the app resolves for
-- the QA Manager (UserMainheadAccess + region inheritance).
SELECT count(*) AS qa_visible_before
FROM "SiteVisit"
WHERE "mainheadId" IN ('<KLB_MAINHEAD_ID>'); -- expected: 1, per audit
```

After

```
-- A1: rows actually changed equals the plan
SELECT count(*) AS rows_backfilled FROM "_backfill_SiteVisit_mainhead_20260604";
-- equals B2.will_backfill

-- A2: NULL volume dropped by exactly rows_backfilled (others remain by design)
SELECT count(*) FILTER (WHERE "mainheadId" IS NULL) AS null_fk_after FROM "SiteVisit";
-- null_fk_after = B1.null_fk_visits - A1.rows_backfilled

-- A3: referential + activity integrity -- must return 0
SELECT count(*) AS broken_or_inactive
FROM "SiteVisit" sv
LEFT JOIN "Mainhead" m ON m.id = sv."mainheadId"
WHERE sv."mainheadId" IS NOT NULL AND (m.id IS NULL OR m."isActive" = FALSE);

-- A4: every backfilled row matches its plan exactly -- must return 0
SELECT count(*) AS diverged
FROM "_backfill_SiteVisit_mainhead_20260604" b
JOIN "SiteVisit" sv ON sv.id = b.site_visit_id
WHERE sv."mainheadId" <> b.assigned_mainhead_id;

-- A5: QA visibility now reflects the data -- re-run B3 with the same id list
SELECT count(*) AS qa_visible_after
FROM "SiteVisit"
WHERE "mainheadId" IN ('<KLB_MAINHEAD_ID>');
-- qa_visible_after = 1 (original) + (# backfilled rows whose text resolved to KLB)
```

Expected reading (anchored to the audit numbers): Admin total stays 7; will_backfill accounts for the legacy NULL rows that resolve cleanly; QA's KL BARAT count rises from 1 to 1 + (KLB-resolved legacy rows). Any residual NULL after backfill is fully explained by ambiguous_skip + unmatched_skip + no_legacy_text — nothing disappears silently.

6. Notes, assumptions & scope boundary

- **Single-tenant assumption.** Mainhead / Organization carry no tenantId; the only tenant anchor is operationalRegion.tenantId (nullable). For multi-tenant safety add to the join: AND m."operationalRegionId" IN (SELECT id FROM "OperationalRegion" WHERE "tenantId" = sv."tenantId") — for Mainheads that have a region; region-less Mainheads need a branch-path policy decision.
- **updatedAt deliberately untouched** to avoid resetting last-activity / operational-health staleness clocks (getLastActivityByVisitId, site-visits.service.ts:1360).
- **Ambiguity is never auto-resolved.** With no code/name uniqueness at the DB level, the script hard-requires exactly one distinct candidate Mainhead; everything else is surfaced for a human.
- **Positive blast radius.** Dashboard counts and defect visibility for QA also scope through siteVisit.mainheadId, so this single backfill restores QA visibility across visits, dashboard, and defects at once.
- **Out of scope (intentionally).** Project.mainheadId and WorkPackage.mainheadId also have parallel legacy text columns, but the QA scope never reads them, so they are not required for QA visibility.

7. Bottom line

The G3 access-control code is correct and unchanged by this work. The backfill is a data correction that populates SiteVisit.mainheadId from the legacy text column for rows that resolve to exactly one active Mainhead. It is idempotent, reversible, and ambiguity-safe; ambiguous and unmatched rows are reported, never guessed. Review the §1 and §2 outputs first, run §3 in a maintenance window, then confirm with the §5 after-queries before archiving the backup table.